

CURSO 2: ALGORITMOS AVANZADOS Y ESTRUCTURAS DE DATOS

CLASE 03 • NIVEL 2 - INTERMEDIO

Clase 03: Tablas Hash y Conjuntos (Sets) para Búsqueda $O(1)$

«Tablas Hash como un Fichero con Índice Alfabético Instantáneo»

Guía de estudio oficial y manual técnico. Diseñado para dominar la programación en Python
mediante modelos mentales rigurosos, diagramas de flujo y código de producción.

Autor & Mentor: William Rodríguez (Wisrovi)

Rol: AI Solutions Architect & Principal Software Engineer

Python: 3.10+ | **Licencia:** MIT | **wisrovi SUITE**

Acerca del Autor y Mentor



William Rodríguez (Wisrovi)

AI Solutions Architect & Principal Software Engineer • Badajoz, España

Ingeniero y arquitecto de software especializado en Inteligencia Artificial Generativa, sistemas multi-agente, Visión por Computador e infraestructuras MLOps de alta disponibilidad. Creador y mantenedor de la suite de software libre wisrovi SUITE en PyPI con más de 26 bibliotecas enfocadas en orquestación de pipelines, caching distribuido y optimización de bases de datos.



GitHub: github.com/wisrovi



LinkedIn: [in/wisrovi-rodriguez](https://in.linkedin.com/in/wisrovi-rodriguez)



DockerHub: hub.docker.com/u/wisrovi



Website: wisrovi.dev

Metodología de Aprendizaje: La Regla de la Bicicleta

En este programa no creemos en el aprendizaje pasivo. Programar no se aprende memorizando manuales o mirando videos en segundo plano mientras tomas café; se aprende **escribiendo código**, enfrentando errores de sintaxis, depurando variables y viendo cómo responde el intérprete en tiempo real.



El Compromiso Activo del Estudiante

Abre Visual Studio Code en cada sesión. Escribe cada ejemplo con tus propias manos. Cambia los números, rompe el código deliberadamente para ver el mensaje de error de Python, y luego arréglalo. Ese proceso de experimentación es el que construye sinapsis duraderas.

Presentación de la Sesión

Bienvenido/a a la **CLASE 03** del **Curso 2: Algoritmos Avanzados y Estructuras de Datos**. Esta guía técnica está estructurada para proporcionarte las bases conceptuales, arquitectónicas y prácticas indispensables para tu progreso.

Objetivos de la Sesión

Competencia Conceptual: Comprender la función hash, manejo de colisiones y costo amortizado $O(1)$.

Competencia Práctica: Resolver problemas de duplicados, intersecciones e indexación asociativa rápida.

Estructura del Documento

Sección	Contenido Temático	Objetivo Pedagógico
Pág 4: Fundamento	Teoría, Modelo Mental y Metáfora	Comprensión del concepto
Pág 5: Flujo	Diagrama de Arquitectura de Memoria	Visualización del flujo interno
Pág 6: Código	Implementación en Python 3.10+	Patrones idiomáticos (PEP 8)
Pág 7: Gotchas	Antipatrones y Trampas Comunes	Prevención de bugs en producción
Pág 8: Conclusiones	Cierre de Sesión & Notas de Repaso	Consolidación del aprendizaje
Pág 9: Bibliografía	Fuentes Canónicas y Desafío	Ampliación y autoestudio

Clase 03: Tablas Hash y Conjuntos (Sets) para Búsqueda O(1)

Las tablas hash convierten una clave arbitraria en un índice numérico mediante una función matemática de hashing.

☀ Metáfora Central: Tablas Hash como un Fichero con Índice Alfabético Instantáneo

Es como un conserje de hotel que sabe instantáneamente el casillero de cada huésped con solo mirar su apellido.

Principios Teóricos y Modelo Mental

Las colisiones ocurren cuando dos claves distintas generan el mismo hash; CPython usa open addressing con perturbación.

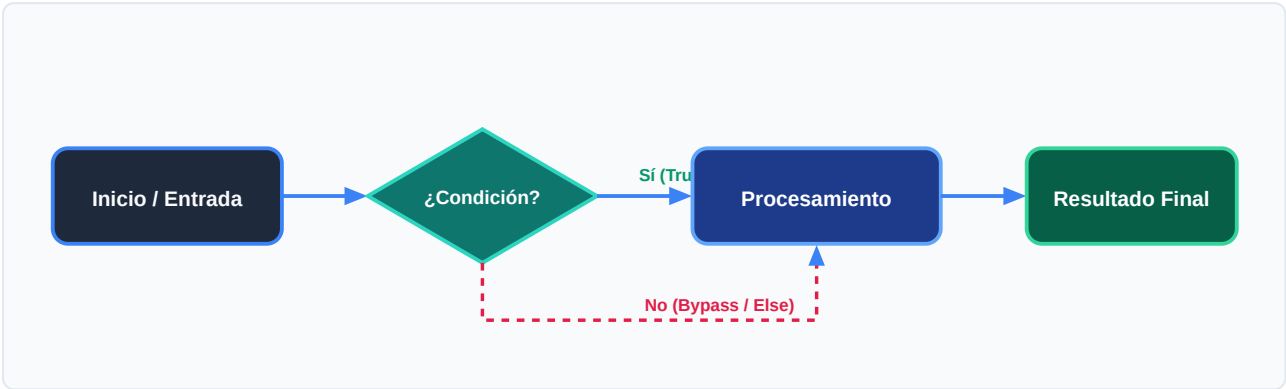
Las operaciones de inserción, búsqueda y eliminación en sets y dicts tienen complejidad temporal promedio O(1).

⚡ Regla de Oro en Python

Cualquier objeto que uses como clave de diccionario o elemento de set debe implementar `__hash__` e inmutabilidad.

Arquitectura y Movimiento de Datos en Memoria

Proceso de hashing, resolución de buckets y deduplicación.



Desglose Paso a Paso del Diagrama

Fase del Flujo	Acción del Intérprete	Estado en Memoria
1. Inicialización	Invocación de hash(objeto).	Firma hash de 64 bits.
2. Evaluación	Mapeo al índice del arreglo de buckets.	Posición en tabla calculada.
3. Transformación	Verificación de igualdad (==) para resolver colisiones.	Elemento exacto encontrado.
4. Retorno / Salida	Inserción o retorno del valor.	Estado de tabla actualizado.

Visualización Mental

Un set es simplemente un diccionario que no almacena valores, solo claves.

Código de Demostración en Python 3.10+

Problema clásico Two-Sum resuelto en $O(n)$ con tabla hash:

main.py (Python 3.10+)

PEP 8 Compliant

```
def two_sum(nums: list[int], target: int) -> tuple[int, int]:
    vistos = {} # mapa: valor -> indice
    for i, num in enumerate(nums):
        complemento = target - num
        if complemento in vistos:
            return (vistos[complemento], i)
        vistos[num] = i
    return (-1, -1)

indices = two_sum([2, 7, 11, 15], 9)
print("Índices que suman 9:", indices) # (0, 1)
```

Análisis de Ingeniería del Código

Uso de un diccionario como tabla hash para reducir una búsqueda $O(n^2)$ a un recorrido lineal $O(n)$.

Buena Práctica de Tipado

El uso sistemático de Type Hints (PEP 484) permite a los editores modernos como VS Code activar autocompletado inteligente y detectar errores antes de la ejecución.

Trampas Frecuentes de Depuración (Gotchas)

Errores de hashabilidad en Python.

⚠ Gotcha Frecuente (Trampa de Principiante)

Intentar usar una lista mutable como clave de diccionario o elemento de set genera `TypeError: unhashable type`.

Comparativa: Antipatrón vs Patrón Recomendado

❌ Antipatrón

```
mi_dict = {}  
mi_dict[[1, 2]] = 'valor' # ❌ TypeError:  
unhashable type: 'list'
```

✅ Patrón Correcto

```
mi_dict = {}  
mi_dict[(1, 2)] = 'valor' # ✅ Tupla inmutable  
hashable
```



Consejo de Resiliencia en Producción

Para agrupar elementos por propiedades, usa `collections.defaultdict(list)`.

Conclusiones Clave de la Sesión

Hemos cubierto los pilares teóricos y prácticos de **Clase 03: Tablas Hash y Conjuntos (Sets) para Búsqueda O(1)**. El dominio de esta unidad te proporciona una base sólida para afrontar la siguiente semana formativa.

Hitos Alcanzados

Comprensión profunda del modelo mental, capacidad de depuración de gotchas comunes y solidez en la sintaxis idiomática de Python 3.10+.

Notas de Repaso Rápido

Concepto	Punto Clave a Recordar
1. Modelo Mental	Tablas Hash como un Fichero con Índice Alfabético Instantáneo
2. Regla de Oro	Cualquier objeto que uses como clave de diccionario o elemento de set debe implementar <code>__hash__</code> e inmutabilidad.
3. Gotcha Crítico	Intentar usar una lista mutable como clave de diccionario o elemento de set genera <code>TypeError: unhashable type</code> .
4. Buenas Prácticas	Para agrupar elementos por propiedades, usa <code>collections.defaultdict(list)</code> .

Mensaje del Instructor

¡Felicitaciones por completar esta lección! Recuerda que la constancia y el pedaleo diario en tu editor de código son los que te convertirán en un programador excepcional.

Fuentes Bibliográficas Oficiales

Para profundizar en los estándares formales del lenguaje y sus implementaciones internas, consulta la documentación canónica:

Recurso Oficial	Descripción	Enlace
Documentación Python 3	Especificación canónica y biblioteca estándar	docs.python.org/3/
PEP 8 — Style Guide	Estándar oficial de formateo y estilo	peps.python.org/pep-0008/
Real Python Tutorials	Patrones de desarrollo e ingeniería	realpython.com
Suite wisrovi en GitHub	Paquetes open source de alto rendimiento	github.com/wisrovi

🏆 Desafío Práctico para el Estudiante

🎯 Reto de la Semana

Implementa una función que encuentre el primer carácter no repetido en una cadena en tiempo $O(n)$.

🔧 Validación Automatizada

Ejecuta `pytest ejercicios/` en tu terminal de VS Code para verificar automáticamente la validez de tu solución.